

Inside the IDE Extension Ecosystem: A Reproducible Manifest-Level Study of 146 VS Code Packages

Prajwal A. K.
Independent Researcher
public artifact reconstruction

August 2026

Abstract

IDE extensions combine user-interface integration with code that may execute in a developer’s environment, yet even basic empirical descriptions of extension package structure are easy to overinterpret. We present a reproducible, execution-free analysis of a preserved July 31, 2026 corpus of 146 content-addressed VSIX packages. All archives contained a parseable public manifest. In this corpus, 124 packages (84.9%) declared a Node entry point, 108 (74.0%) contributed commands, 40 (27.4%) contributed views, and 12 (8.2%) declared wildcard activation. Only 14 manifests (9.6%) explicitly represented untrusted-workspace behavior and four (2.7%) represented virtual-workspace behavior. Repository or homepage metadata appeared in 125 packages (85.6%), while 90 (61.6%) declared a license. These are manifest-level measurements, not observations of runtime behavior or security. We publish the study protocol, content hashes, derived feature table, aggregation code, and arXiv-ready sources. Because the original corpus-selection log was not recoverable, we treat the collection as a convenience snapshot and make no marketplace-wide prevalence claims.

1 Introduction

Visual Studio Code extensions are ZIP-based VSIX packages whose manifests declare compatibility, activation, executable entry points, and contributions to the host interface. The platform exposes APIs for commands, language services, debugging, authentication, terminals, webviews, and workspace-aware operation [1, 2, 4]. This flexibility is central to the editor ecosystem, but it complicates empirical description: a declaration indicates that a package asks the host to integrate a feature; it does not show how the feature behaves.

This paper asks what can be learned reproducibly from public package manifests alone. Our contributions are:

- an execution-free measurement procedure over 146 content-addressed VSIX archives;
- descriptive results for entry points, activation, contribution points, provenance fields, and workspace declarations;
- an artifact that records every operational definition and every limitation needed to avoid turning package

metadata into unsupported security conclusions.

The work intentionally contains no product evaluation, scanner result, malware label, or claim about extension safety. Its unit of evidence is the archived manifest.

2 Background

An extension manifest identifies the package and specifies an engine range. A `main` field selects a Node extension-host entry point, while `browser` can identify a web-extension entry point [1, 6]. Activation events describe when an extension becomes eligible for activation; modern VS Code versions can infer some activation from contribution declarations [2]. The `contributes` object statically declares host integrations such as commands, languages, debuggers, views, task definitions, and custom editors [3].

Workspace capability declarations allow authors to explain support for untrusted or virtual workspaces [4, 5]. Their presence is useful documentation, but their absence is not itself evidence of a defect. Similarly, repository, homepage, and license fields improve traceability without proving behavior. We preserve these distinctions throughout.

3 Method

3.1 Corpus and unit of analysis

The input is a surviving local snapshot, 20260731-full, containing 146 VSIX files named by SHA-256 digest and totaling 500,990,340 compressed bytes. A separate 20-file pilot directory was excluded to avoid duplicated packages. Every VSIX directly inside the full directory was included; all 146 yielded a valid `extension/package.json` manifest.

The original marketplace query, ranking, and inclusion log were lost with an untracked research directory. We therefore classify the corpus as a convenience snapshot. It is neither a census nor a probability sample, and our percentages describe only these 146 archives.

3.2 Extraction

The artifact script independently hashes each archive, opens it as ZIP, and reads only its public manifest. It

Table 1: Execution and activation declarations ($n = 146$).

Feature	Count	Percent
Node <code>main</code> entry point	124	84.9
Browser entry point	10	6.8
Explicit activation event(s)	111	76.0
Wildcard activation	12	8.2

does not import or execute packaged code and makes no network request. For each package, it records identity and version; compressed size; engine constraint; Node and browser entry points; activation-event count and wildcard activation; contribution counts; direct manifest dependency counts; repository/homepage and license presence; and explicit untrusted- and virtual-workspace capability objects.

Boolean fields are reported as counts and percentages. Count fields are reported by median, maximum, and nonzero frequency. No inferential test is applied because the sample design does not support population inference. The full protocol defines each field and preserves extraction failures rather than silently discarding them.

4 Results

4.1 Execution and activation declarations

Table 1 reports the entry-point and activation structure. Most packages declared a Node entry point (124, 84.9%); ten (6.8%) declared a browser entry point. The categories overlap. Activation events were explicitly present in 111 manifests (76.0%), with a median of one and maximum of 21. Twelve packages (8.2%) used wildcard activation.

These values do not establish when code actually ran. In particular, contribution-based activation inference means a zero explicit-event count is not equivalent to a never-activated extension [2].

4.2 Host contribution surface

Commands were the most common measured contribution: 108 packages (74.0%), with a median of two and a maximum of 172. Forty packages (27.4%) contributed one or more views (maximum 19); 27 (18.5%) declared languages. Debuggers and custom editors each appeared in six packages (4.1%); one package declared a task definition and one declared authentication. Table 2 summarizes the results.

A contribution is a structural affordance. Command count does not measure command sensitivity; view count does not establish webview content; language and debugger declarations do not measure implementation quality.

4.3 Provenance and workspace declarations

Repository or homepage metadata appeared in 125 manifests (85.6%); 90 (61.6%) declared a li-

Table 2: Nonzero contribution declarations.

Contribution	Count	Percent	Maximum
Commands	108	74.0	172
Views	40	27.4	19
Languages	27	18.5	16
Debuggers	6	4.1	1
Custom editors	6	4.1	2
Task definitions	1	0.7	1
Authentication	1	0.7	1

cense. Fourteen (9.6%) contained an object under `capabilities.untrustedWorkspaces`, and four (2.7%) did so for `virtualWorkspaces`. Direct runtime dependencies were declared by 72 packages (49.3%), with a median of zero and maximum of 89. This manifest count omits bundled and transitive code and is therefore not a software-composition inventory.

5 Discussion

The corpus demonstrates that extension manifests expose a useful but bounded structural layer. Entry points are common, while contribution profiles vary substantially. Provenance URLs are frequent but not universal. Explicit workspace capability objects are uncommon in this snapshot, though platform defaults and historical engine ranges prevent a simple quality interpretation.

The methodological lesson is more important than any individual percentage. Public manifest fields can support reproducible questions about declared integration. They cannot, alone, establish maliciousness, safety, effective privilege use, or runtime data flow. Reviews that collapse popularity, publisher identity, contribution count, or activation style into a verdict exceed the evidence available at this layer.

For ecosystem research, future collections should preserve the registry request, ordering, inclusion criteria, collection timestamp per package, and response metadata alongside content hashes. Longitudinal snapshots could then separate ecosystem change from sampling change. Runtime questions would require a separate protocol, isolation boundary, and validation design.

6 Threats to Validity

Selection. The original sampling log is unavailable. The corpus may overrepresent particular ranks, categories, registries, or package types. Results must not be generalized to all extensions.

Construct. Manifest declarations are proxies for package structure, not executed behavior. Defaults, inferred activation, generated manifests, bundled code, and dynamic behavior can make declarations incomplete.

Temporal. The data form one snapshot. Marketplace packages can be updated, unpublished, or replaced, so this study establishes no trend.

Extraction. JSON shape varies. The artifact handles list- and map-shaped contribution collections and records

failures. Independent recomputation from hashes and inspection of the feature CSV remain advisable.

7 Artifact Availability

The companion artifact includes the complete extraction script, study protocol, per-archive derived CSV, aggregate JSON, manuscript source, and an arXiv-ready source archive. Raw VSIX files are not redistributed because package licenses differ. Reproduction requires access to the preserved content-addressed corpus. The artifact is designed so that recalculation changes are visible as ordinary text diffs.

8 Conclusion

We reconstructed a conservative empirical view of 146 archived IDE extension packages using only public manifests. The results quantify declared entry points, activation, host contributions, provenance metadata, dependencies, and workspace compatibility while preserving a strict boundary between declarations and behavior. That boundary—together with content addressing, explicit sampling limits, and executable analysis—is the central contribution.

References

- [1] Microsoft, “Extension Anatomy,” Visual Studio Code Documentation. <https://code.visualstudio.com/api/get-started/extension-anatomy>
- [2] Microsoft, “Activation Events,” Visual Studio Code Documentation. <https://code.visualstudio.com/api/references/activation-events>
- [3] Microsoft, “Contribution Points,” Visual Studio Code Documentation. <https://code.visualstudio.com/api/references/contribution-points>
- [4] Microsoft, “Extension Manifest: Capabilities,” Visual Studio Code Documentation. <https://code.visualstudio.com/api/references/extension-manifest>
- [5] Microsoft, “Workspace Trust Extension Guide,” Visual Studio Code Documentation. <https://code.visualstudio.com/api/extension-guides/workspace-trust>
- [6] Microsoft, “Web Extensions,” Visual Studio Code Documentation. <https://code.visualstudio.com/api/extension-guides/web-extensions>